

Informe de laboratorio 03

Tema: Divide y vencerás Ordenamiento: MergeSort y Quicksort.

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
03	Divide y vencerás Ordenamiento: MergeSort y Quicksort.	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	17/09/2026	21/09/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Implementa Mergesort para ordenar una lista de registros de estudiantes (código: string, nombre: string, promedio ponderado: float) por promedio ponderado, ingresados por el usuario.

Listing 1: Ejercicio 01: MergeSort ordena lista de registros por promedio ponderado

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 struct Estudiante {
7     string codigo;
8     string nombre;
9     float promedio;
10 };
11
12 void merge(vector<Estudiante>& arr, int izq, int mid, int der) {
13     vector<Estudiante> left(arr.begin() + izq, arr.begin() + mid + 1);
14     vector<Estudiante> right(arr.begin() + mid + 1, arr.begin() + der + 1);
15 }
```

```
16  int i = 0, j = 0, k = izq;
17  while (i < left.size() && j < right.size()) {
18      if (left[i].promedio <= right[j].promedio) {
19          arr[k++] = left[i++];
20      } else {
21          arr[k++] = right[j++];
22      }
23  }
24  while (i < left.size()) arr[k++] = left[i++];
25  while (j < right.size()) arr[k++] = right[j++];
26  }
27
28  void mergesort(vector<Estudiante>& arr, int izq, int der) {
29      if (izq < der) {
30          int mid = (izq + der) / 2;
31          mergesort(arr, izq, mid);
32          mergesort(arr, mid + 1, der);
33          merge(arr, izq, mid, der);
34      }
35  }
36
37  int main() {
38      int n;
39      cout << "Ingrese cantidad de estudiantes: ";
40      cin >> n;
41
42      vector<Estudiante> estudiantes(n);
43      for (int i = 0; i < n; i++) {
44          cout << "\nEstudiante " << i + 1 << ": " << endl;
45          cout << "Codigo: "; cin >> estudiantes[i].codigo;
46          cout << "Nombre: "; cin >> estudiantes[i].nombre;
47          cout << "Promedio: "; cin >> estudiantes[i].promedio;
48      }
49
50      mergesort(estudiantes, 0, n - 1);
51
52      cout << "\nEstudiantes ordenados por promedio:\n";
53      cout << "Codigo\t\tNombre\t\tPromedio\n";
54      cout << "-----\n";
55      for (auto& e : estudiantes) {
56          cout << e.codigo << "\t\t" << e.nombre << "\t\t" << e.promedio << "\n";
57      }
58
59      return 0;
60  }
```

- Primero se solicita la cantidad de estudiantes y se ingresan sus datos, los cuales se almacenan en un vector. Luego, MergeSort divide el vector en dos partes pequeñas de forma repetitiva hasta que cada parte contiene solo un estudiante. Después compara los promedios de los estudiantes de cada parte y los une nuevamente en orden, colocando primero al que tenga el promedio más bajo. Al finalizar, el programa muestra todos los estudiantes ordenados por promedio. Este algoritmo es eficiente porque puede ordenar muchos datos con una complejidad de $O(n \log n)$.

```

camiloasd .../labADA/lab03/e-n □ main !? ▼ 08:54 g++ e1.cpp -o e1

camiloasd .../labADA/lab03/e-n □ main !? ▼ 09:07 ./e1
Ingrese cantidad de estudiantes: 4

Estudiante 1:
Codigo: a1s3
Nombre: Camilo
Promedio: 19.5

Estudiante 2:
Codigo: a2s3
Nombre: Bladimir
Promedio: 18.12

Estudiante 3:
Codigo: a3s3
Nombre: Alexander
Promedio: 14.22

Estudiante 4:
Codigo: a4s3
Nombre: Sofia
Promedio: 17.66

Estudiantes ordenados por promedio:
Codigo      Nombre      Promedio
-----
a3s3        Alexander   14.22
a4s3        Sofia       17.66
a2s3        Bladimir   18.12
a1s3        Camilo      19.5

camiloasd .../labADA/lab03/e-n □ main !? ▼ 09:09

```

Figura 1: Ejecución de e1.cpp

1.2. Ejercicio 2

Implementa Quicksort sobre un arreglo de números flotantes. Analiza su rendimiento con pivote fijo y con pivote aleatorio, y explica en qué casos cada estrategia de pivote es más eficiente.

Listing 2: Ejercicio 02: Quicksort sobre numeros float

```

1 #include <iostream>
2 #include <vector>
3 #include <chrono>
4 #include <cstdlib>
5 #include <ctime>
6 using namespace std;
7
8 int particionFija(vector<float>& arr, int izq, int der) {
9     float pivot = arr[der];
10    int i = izq - 1;

```

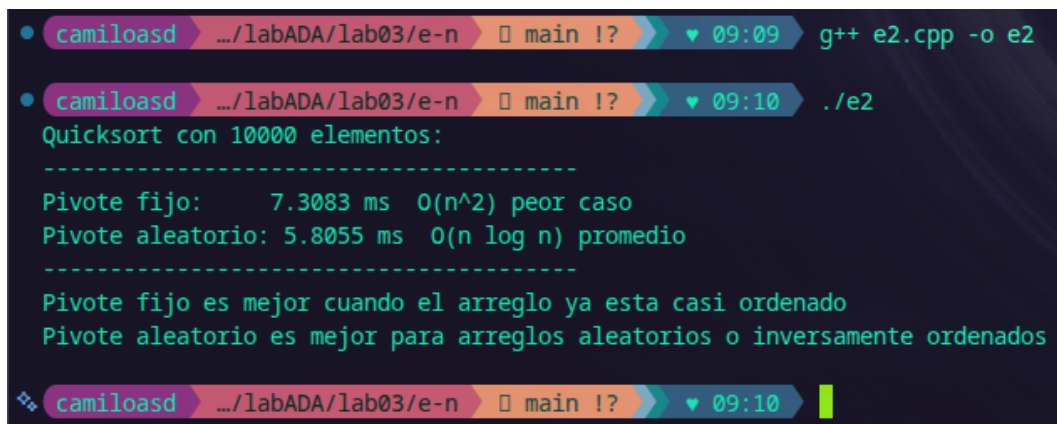
```
11     for (int j = izq; j < der; j++) {
12         if (arr[j] <= pivot) {
13             swap(arr[++i], arr[j]);
14         }
15     }
16     swap(arr[i + 1], arr[der]);
17     return i + 1;
18 }
19
20 void quicksortFijo(vector<float>& arr, int izq, int der) {
21     if (izq < der) {
22         int pi = particionFija(arr, izq, der);
23         quicksortFijo(arr, izq, pi - 1);
24         quicksortFijo(arr, pi + 1, der);
25     }
26 }
27
28 // Pivote aleatorio
29 int particionAleatoria(vector<float>& arr, int izq, int der) {
30     int pivotIdx = izq + rand() % (der - izq + 1);
31     swap(arr[pivotIdx], arr[der]);
32     return particionFija(arr, izq, der);
33 }
34
35 void quicksortAleatorio(vector<float>& arr, int izq, int der) {
36     if (izq < der) {
37         int pi = particionAleatoria(arr, izq, der);
38         quicksortAleatorio(arr, izq, pi - 1);
39         quicksortAleatorio(arr, pi + 1, der);
40     }
41 }
42
43 int main() {
44     srand(time(0));
45     int n = 10000;
46
47     vector<float> arr1(n), arr2(n);
48     for (int i = 0; i < n; i++) {
49         arr1[i] = arr2[i] = (float)(rand() % 10000) / 100.0;
50     }
51
52     // Pivote fijo
53     auto inicio1 = chrono::high_resolution_clock::now();
54     quicksortFijo(arr1, 0, n - 1);
55     auto fin1 = chrono::high_resolution_clock::now();
56     chrono::duration<double, milli> tiempo1 = fin1 - inicio1;
57
58     // Pivote aleatorio
59     auto inicio2 = chrono::high_resolution_clock::now();
60     quicksortAleatorio(arr2, 0, n - 1);
61     auto fin2 = chrono::high_resolution_clock::now();
62     chrono::duration<double, milli> tiempo2 = fin2 - inicio2;
63
64     cout << "Quicksort con " << n << " elementos:\n";
65     cout << "-----\n";
66     cout << "Pivote fijo:  " << tiempo1.count() << " ms O(n^2) peor caso\n";
```

```

67 cout << "Pivote aleatorio: " << tiempo2.count() << " ms O(n log n) promedio\n";
68 cout << "-----\n";
69 cout << "Pivote fijo es mejor cuando el arreglo ya esta casi ordenado\n";
70 cout << "Pivote aleatorio es mejor para arreglos aleatorios o inversamente ordenados\n";
71
72 return 0;
73 }

```

- La primera forma utiliza un pivote fijo, que siempre es el último elemento del arreglo, mientras que la segunda selecciona un pivote aleatorio. QuickSort funciona eligiendo un pivote y separando los valores menores o iguales a un lado y los mayores al otro; luego repite el proceso en cada parte hasta ordenar todo el arreglo. El programa mide el tiempo que tarda cada versión y muestra los resultados en milisegundos. El uso de un pivote aleatorio ayuda a evitar divisiones desbalanceadas y reduce la posibilidad de que QuickSort alcance su peor caso, cuya complejidad es $O(n^2)$; normalmente, ambas versiones tienen un rendimiento promedio de $O(n \log n)$.



```

camiloasd .../labADA/lab03/e-n [main !?] 09:09 g++ e2.cpp -o e2
camiloasd .../labADA/lab03/e-n [main !?] 09:10 ./e2
Quicksort con 10000 elementos:
-----
Pivote fijo:      7.3083 ms  O(n^2) peor caso
Pivote aleatorio: 5.8055 ms  O(n log n) promedio
-----
Pivote fijo es mejor cuando el arreglo ya esta casi ordenado
Pivote aleatorio es mejor para arreglos aleatorios o inversamente ordenados
camiloasd .../labADA/lab03/e-n [main !?] 09:10

```

Figura 2: Ejecución de e2.cpp

1.3. Ejercicio 3

Genera un arreglo de n elementos, ordénalo con Mergesort o Quicksort, implementa una búsqueda binaria para encontrar un elemento dentro del arreglo resultante, y calcula la complejidad temporal total del proceso (ordenamiento + búsqueda) comparándola con la búsqueda secuencial sin ordenamiento previo.

Listing 3: Ejercicio 03: Mergesort, Quicksort y busqueda binaria

```

1 #include <iostream>
2 #include <vector>
3 #include <chrono>
4 #include <cstdlib>
5 using namespace std;
6
7 void merge(vector<int>& arr, int izq, int mid, int der) {
8     vector<int> left(arr.begin() + izq, arr.begin() + mid + 1);
9     vector<int> right(arr.begin() + mid + 1, arr.begin() + der + 1);
10    int i = 0, j = 0, k = izq;
11    while (i < left.size() && j < right.size())

```

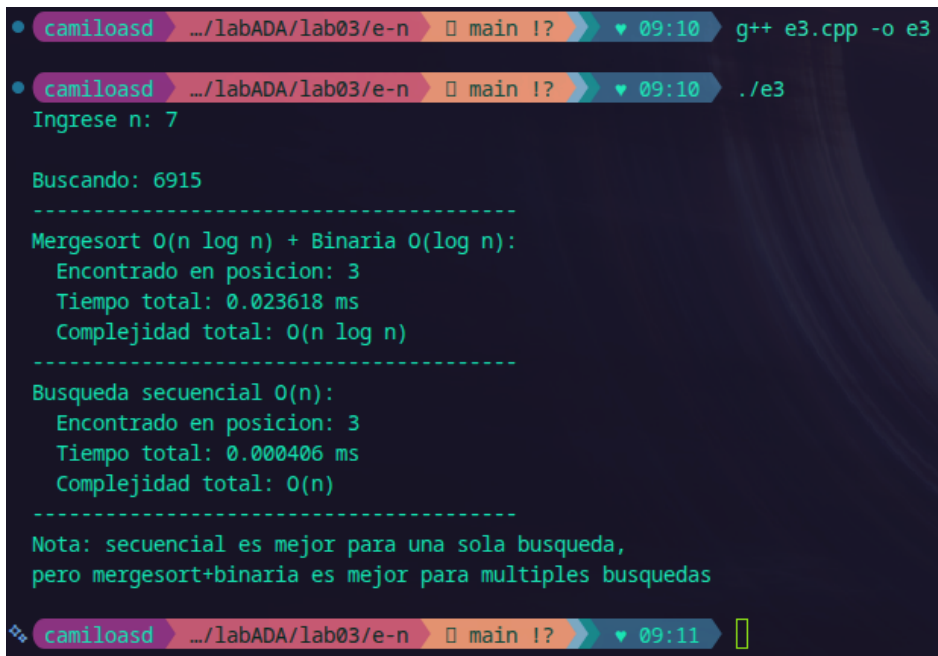
```
12     arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
13     while (i < left.size()) arr[k++] = left[i++];
14     while (j < right.size()) arr[k++] = right[j++];
15 }
16
17 void mergesort(vector<int>& arr, int izq, int der) {
18     if (izq < der) {
19         int mid = (izq + der) / 2;
20         mergesort(arr, izq, mid);
21         mergesort(arr, mid + 1, der);
22         merge(arr, izq, mid, der);
23     }
24 }
25
26 int busquedaBinaria(vector<int>& arr, int objetivo) {
27     int izq = 0, der = arr.size() - 1;
28     while (izq <= der) {
29         int mid = (izq + der) / 2;
30         if (arr[mid] == objetivo) return mid;
31         else if (arr[mid] < objetivo) izq = mid + 1;
32         else der = mid - 1;
33     }
34     return -1;
35 }
36
37 int busquedaSecuencial(vector<int>& arr, int objetivo) {
38     for (int i = 0; i < arr.size(); i++)
39         if (arr[i] == objetivo) return i;
40     return -1;
41 }
42
43 int main() {
44     int n;
45     cout << "Ingrese n: ";
46     cin >> n;
47
48     vector<int> arr(n), arrOriginal(n);
49     for (int i = 0; i < n; i++)
50         arr[i] = arrOriginal[i] = rand() % 10000;
51
52     int objetivo = arr[n / 2];
53
54     // Mergesort + binaria
55     auto inicio1 = chrono::high_resolution_clock::now();
56     mergesort(arr, 0, n - 1);
57     int pos1 = busquedaBinaria(arr, objetivo);
58     auto fin1 = chrono::high_resolution_clock::now();
59     chrono::duration<double, milli> tiempo1 = fin1 - inicio1;
60
61     // Secuencial sin ordenar
62     auto inicio2 = chrono::high_resolution_clock::now();
63     int pos2 = busquedaSecuencial(arrOriginal, objetivo);
64     auto fin2 = chrono::high_resolution_clock::now();
65     chrono::duration<double, milli> tiempo2 = fin2 - inicio2;
66
67     cout << "\nBuscando: " << objetivo << endl;
```

```

68 cout << "-----\n";
69 cout << "Mergesort O(n log n) + Binaria O(log n):\n";
70 cout << " Encontrado en posicion: " << pos1 << endl;
71 cout << " Tiempo total: " << tiempo1.count() << " ms\n";
72 cout << " Complejidad total: O(n log n)\n";
73 cout << "-----\n";
74 cout << "Busqueda secuencial O(n):\n";
75 cout << " Encontrado en posicion: " << pos2 << endl;
76 cout << " Tiempo total: " << tiempo2.count() << " ms\n";
77 cout << " Complejidad total: O(n)\n";
78 cout << "-----\n";
79 cout << "Nota: secuencial es mejor para una sola busqueda,\n";
80 cout << "pero mergesort+binaria es mejor para multiples busquedas\n";
81
82 return 0;
83 }

```

- Primero genera un arreglo de tamaño n y guarda una copia sin ordenar. Luego selecciona como objetivo un valor ubicado aproximadamente en la mitad del arreglo. La primera estrategia ordena los datos con MergeSort y después busca el número mediante búsqueda binaria. MergeSort tiene complejidad $O(n \log n)$ y la búsqueda binaria $O(\log n)$, pero como es necesario ordenar antes, la complejidad total es $O(n \log n)$. La segunda estrategia realiza una búsqueda secuencial sobre el arreglo original, revisando los elementos uno por uno hasta encontrar el objetivo, con complejidad $O(n)$. El programa mide el tiempo de ambas estrategias y muestra las posiciones encontradas. Para una sola búsqueda, la búsqueda secuencial suele ser más conveniente porque no necesita ordenar; sin embargo, si se realizarán muchas búsquedas sobre los mismos datos, conviene ordenar una vez con MergeSort y luego usar búsqueda binaria.



```

camiloasd .../labADA/lab03/e-n main !? 09:10 g++ e3.cpp -o e3
camiloasd .../labADA/lab03/e-n main !? 09:10 ./e3
Ingrese n: 7

Buscando: 6915
-----
Mergesort O(n log n) + Binaria O(log n):
  Encontrado en posicion: 3
  Tiempo total: 0.023618 ms
  Complejidad total: O(n log n)
-----
Busqueda secuencial O(n):
  Encontrado en posicion: 3
  Tiempo total: 0.000406 ms
  Complejidad total: O(n)
-----
Nota: secuencial es mejor para una sola busqueda,
pero mergesort+binaria es mejor para multiples busquedas
camiloasd .../labADA/lab03/e-n main !? 09:11

```

Figura 3: Ejecución de e3.cpp

1.4. Ejercicio 4

Ordena 10 000 elementos aleatorios y compara los tiempos de ejecución de Mergesort, Quicksort, InsertionSort y Selection Sort.

Listing 4: Ejercicio 04: Tiempos de ejecución de Mergesort, Quicksort, InsertionSort y Selection Sort

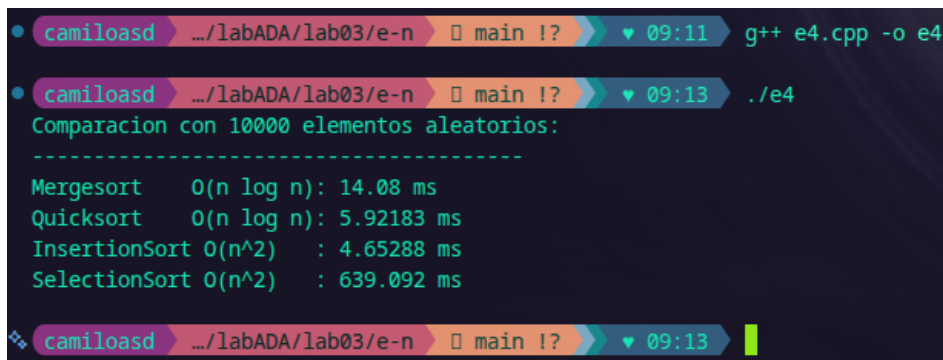
```
1 #include <iostream>
2 #include <vector>
3 #include <chrono>
4 #include <cstdlib>
5 #include <algorithm>
6 using namespace std;
7
8 void mergesortHelper(vector<int>& arr, int izq, int der) {
9     if (izq < der) {
10         int mid = (izq + der) / 2;
11         mergesortHelper(arr, izq, mid);
12         mergesortHelper(arr, mid + 1, der);
13         vector<int> left(arr.begin() + izq, arr.begin() + mid + 1);
14         vector<int> right(arr.begin() + mid + 1, arr.begin() + der + 1);
15         int i = 0, j = 0, k = izq;
16         while (i < left.size() && j < right.size())
17             arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
18         while (i < left.size()) arr[k++] = left[i++];
19         while (j < right.size()) arr[k++] = right[j++];
20     }
21 }
22
23 void quicksortHelper(vector<int>& arr, int izq, int der) {
24     if (izq < der) {
25         int pivot = arr[der], i = izq - 1;
26         for (int j = izq; j < der; j++)
27             if (arr[j] <= pivot) swap(arr[++i], arr[j]);
28         swap(arr[i + 1], arr[der]);
29         int pi = i + 1;
30         quicksortHelper(arr, izq, pi - 1);
31         quicksortHelper(arr, pi + 1, der);
32     }
33 }
34
35 void insertionSort(vector<int>& arr) {
36     for (int i = 1; i < arr.size(); i++) {
37         int key = arr[i], j = i - 1;
38         while (j >= 0 && arr[j] > key) arr[j + 1] = arr[j--];
39         arr[j + 1] = key;
40     }
41 }
42
43 void selectionSort(vector<int>& arr) {
44     for (int i = 0; i < arr.size() - 1; i++) {
45         int minIdx = i;
46         for (int j = i + 1; j < arr.size(); j++)
47             if (arr[j] < arr[minIdx]) minIdx = j;
48         swap(arr[i], arr[minIdx]);
49     }
50 }
```

```

51
52 int main() {
53     int n = 10000;
54     vector<int> base(n);
55     for (int i = 0; i < n; i++) base[i] = rand() % 100000;
56
57     auto medir = [&](string nombre, auto func) {
58         vector<int> arr = base;
59         auto inicio = chrono::high_resolution_clock::now();
60         func(arr);
61         auto fin = chrono::high_resolution_clock::now();
62         chrono::duration<double, milli> tiempo = fin - inicio;
63         cout << nombre << ": " << tiempo.count() << " ms\n";
64     };
65
66     cout << "Comparacion con " << n << " elementos aleatorios:\n";
67     cout << "-----\n";
68     medir("Mergesort  O(n log n)", [](vector<int>& a){ mergesortHelper(a, 0, a.size()-1); });
69     medir("Quicksort  O(n log n)", [](vector<int>& a){ quicksortHelper(a, 0, a.size()-1); });
70     medir("InsertionSort O(n^2) ", [](vector<int>& a){ insertionSort(a); });
71     medir("SelectionSort O(n^2) ", [](vector<int>& a){ selectionSort(a); });
72
73     return 0;
74 }

```

- Para cada algoritmo se crea una copia del mismo arreglo y se mide cuánto tarda en ordenarlo. MergeSort y QuickSort suelen ser más rápidos porque tienen complejidad promedio de $O(n \log n)$, mientras que InsertionSort y SelectionSort tienen complejidad $O(n^2)$, por lo que tardan más con muchos elementos.



```

camiloasd .../labADA/lab03/e-n □ main !? ▼ 09:11 g++ e4.cpp -o e4
camiloasd .../labADA/lab03/e-n □ main !? ▼ 09:13 ./e4
Comparacion con 10000 elementos aleatorios:
-----
Mergesort  O(n log n): 14.08 ms
Quicksort  O(n log n): 5.92183 ms
InsertionSort O(n^2) : 4.65288 ms
SelectionSort O(n^2) : 639.092 ms
camiloasd .../labADA/lab03/e-n □ main !? ▼ 09:13

```

Figura 4: Ejecución de e4.cpp

1.5. Ejercicio 5

Desarrolla una aplicación en C++ que lea un archivo .txt con productos (código, nombre, precio) y los ordene de mayor a menor precio con ambos algoritmos.

Listing 5: Ejercicio 05: Aplicación en C++numbers

```

#include <iostream>
#include <fstream>

```

```
#include <vector>
#include <string>
#include <chrono>
using namespace std;

struct Producto {
    string codigo;
    string nombre;
    float precio;
};

vector<Producto> leerArchivo(string ruta) {
    vector<Producto> productos;
    ifstream archivo(ruta);
    if (!archivo.is_open()) {
        cout << "Error al abrir el archivo\n";
        return productos;
    }
    string linea;
    while (getline(archivo, linea)) {
        stringstream ss(linea);
        Producto p;
        string precioStr;

        getline(ss, p.codigo, ',');
        getline(ss, p.nombre, ',');
        getline(ss, precioStr, ',');

        p.precio = stof(precioStr); // Convierte texto a float
        productos.push_back(p);
    }
    archivo.close();
    return productos;
}

// Mergesort
void mergeProductos(vector<Producto>& arr, int izq, int mid, int der) {
    vector<Producto> left(arr.begin() + izq, arr.begin() + mid + 1);
    vector<Producto> right(arr.begin() + mid + 1, arr.begin() + der + 1);
    int i = 0, j = 0, k = izq;
    while (i < left.size() && j < right.size())
        arr[k++] = (left[i].precio >= right[j].precio) ? left[i++] : right[j++];
    while (i < left.size()) arr[k++] = left[i++];
    while (j < right.size()) arr[k++] = right[j++];
}

void mergesortProductos(vector<Producto>& arr, int izq, int der) {
    if (izq < der) {
        int mid = (izq + der) / 2;
        mergesortProductos(arr, izq, mid);
        mergesortProductos(arr, mid + 1, der);
        mergeProductos(arr, izq, mid, der);
    }
}

// Quicksort
```

```
int particionProductos(vector<Producto>& arr, int izq, int der) {
    float pivot = arr[der].precio;
    int i = izq - 1;
    for (int j = izq; j < der; j++)
        if (arr[j].precio >= pivot) swap(arr[++i], arr[j]);
    swap(arr[i + 1], arr[der]);
    return i + 1;
}

void quicksortProductos(vector<Producto>& arr, int izq, int der) {
    if (izq < der) {
        int pi = particionProductos(arr, izq, der);
        quicksortProductos(arr, izq, pi - 1);
        quicksortProductos(arr, pi + 1, der);
    }
}

void imprimirProductos(vector<Producto>& arr) {
    cout << "Codigo\t\tNombre\t\tPrecio\n";
    cout << "-----\n";
    for (auto& p : arr)
        cout << p.codigo << "\t\t" << p.nombre << "\t\t" << p.precio << "\n";
}

int main() {
    vector<Producto> productos1 = leerArchivo("productos.txt");
    vector<Producto> productos2 = productos1;

    if (productos1.empty()) return 1;

    // Mergesort
    auto inicio1 = chrono::high_resolution_clock::now();
    mergesortProductos(productos1, 0, productos1.size() - 1);
    auto fin1 = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> tiempo1 = fin1 - inicio1;

    cout << "Ordenado con Mergesort (mayor a menor precio):\n";
    imprimirProductos(productos1);
    cout << "Tiempo: " << tiempo1.count() << " ms\n\n";

    // Quicksort
    auto inicio2 = chrono::high_resolution_clock::now();
    quicksortProductos(productos2, 0, productos2.size() - 1);
    auto fin2 = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> tiempo2 = fin2 - inicio2;

    cout << "Ordenado con Quicksort (mayor a menor precio):\n";
    imprimirProductos(productos2);
    cout << "Tiempo: " << tiempo2.count() << " ms\n\n";

    return 0;
}
```

- Lee los datos de productos desde el archivo productos.txt, donde cada producto contiene código, nombre y precio. Luego ordena los productos de mayor a menor precio utilizando dos algoritmos: MergeSort y QuickSort. El programa mide el tiempo que tarda cada uno, muestra la lista

ordenada y permite comparar su rendimiento al trabajar con los mismos datos.

```
lab03 > e-n >  productos.txt
1  P001,Laptop Gamer,3500.00
2  P002,Mouse Inalambrico,45.50
3  P003,Teclado Mecanico,120.00
4  P004,Monitor 24 Pulgadas,850.50
5  P005,Audifonos Bluetooth,199.90
6  P006,Silla Gamer,950.00
7  P007,Disco SSD 1TB,320.00
8  P008,Memoria RAM 16GB,210.00
9  P009,Tarjeta Grafica,2800.00
10 P010,Procesador i7,1450.00
11 P011,Impresora Multifuncional,450.00
12 P012,Webcam HD,130.00
13 P013,Alfombrilla Mouse,25.00
14 P014,Router Wi-Fi 6,290.00
15 P015,Parlantes USB,65.00
```

Figura 5: productos.txt

```

camiloasd .../labADA/lab03/e-n main !? 09:13 g++ e5.cpp -o e5

camiloasd .../labADA/lab03/e-n main !? 09:14 ./e5
Ordenado con Mergesort (mayor a menor precio):
Codigo      Nombre      Precio
-----
P001        Laptop Gamer      3500
P009        Tarjeta Grafica    2800
P010        Procesador i7      1450
P006        Silla Gamer        950
P004        Monitor 24 Pulgadas 850.5
P011        Impresora Multifuncional 450
P007        Disco SSD 1TB      320
P014        Router Wi-Fi 6     290
P008        Memoria RAM 16GB   210
P005        Audifonos Bluetooth 199.9
P012        Webcam HD          130
P003        Teclado Mecanico   120
P015        Parlantes USB       65
P002        Mouse Inalambrico  45.5
P013        Alfombrilla Mouse  25
Tiempo: 0.051064 ms

Ordenado con Quicksort (mayor a menor precio):
Codigo      Nombre      Precio
-----
P001        Laptop Gamer      3500
P009        Tarjeta Grafica    2800
P010        Procesador i7      1450
P006        Silla Gamer        950
P004        Monitor 24 Pulgadas 850.5
P011        Impresora Multifuncional 450
P007        Disco SSD 1TB      320
P014        Router Wi-Fi 6     290
P008        Memoria RAM 16GB   210
P005        Audifonos Bluetooth 199.9
P012        Webcam HD          130
P003        Teclado Mecanico   120
P015        Parlantes USB       65
P002        Mouse Inalambrico  45.5
P013        Alfombrilla Mouse  25
Tiempo: 0.036993 ms

camiloasd .../labADA/lab03/e-n main !? 09:14

```

Figura 6: Ejecución de e5.cpp

2. Cuestionario

1. La estrategia Divide y vencerás se diferencia de los algoritmos simples porque divide el problema en partes más pequeñas, resuelve cada una por separado y luego combina los resultados. En cambio, algoritmos como InsertionSort o SelectionSort trabajan de forma más directa y suelen requerir más comparaciones o movimientos en casos grandes, por lo que son menos eficientes en volumen de datos.
2. Mergesort siempre tiene complejidad $O(n \log n)$ porque, sin importar el arreglo inicial, se divide repetidamente en dos mitades hasta llegar a subproblemas de tamaño 1 y luego realiza una fusión lineal en cada nivel. Aunque el número de niveles es $\log n$, la cantidad de trabajo por nivel es $O(n)$, por lo que el costo total se mantiene estable en todos los casos.
3. Quicksort alcanza su peor caso de $O(n^2)$ cuando el pivote elegido siempre divide el arreglo de forma muy desbalanceada, por ejemplo cuando el arreglo ya está ordenado o en orden inverso y se toma como pivote el primer o el último elemento. En ese escenario, una parte queda con casi todos los elementos y la otra con pocos, lo que genera muchas llamadas recursivas innecesarias.
4. En un entorno con restricciones de memoria, Mergesort suele ser menos conveniente si se implementa con copias adicionales durante la fusión, porque necesita memoria auxiliar. QuickSort, al ordenar in-place en muchos casos, resulta preferible cuando el objetivo es ahorrar memoria; sin embargo, si la estabilidad del orden es importante, MergeSort puede ser más apropiado.
5. Para mejorar el rendimiento práctico de Quicksort se pueden usar pivotes más equilibrados, como el pivote medio o aleatorio, y además aplicar optimizaciones como la ordenación de arreglos pequeños con InsertionSort. Estas estrategias reducen el riesgo de casos desbalanceados y mejoran la velocidad real en entradas grandes.

3. Conclusiones

- En esta práctica se comprobó que MergeSort y QuickSort son más adecuados para grandes volúmenes de datos que los algoritmos de ordenamiento simple, ya que ambos tienen complejidad promedio de $O(n \log n)$. Esto se refleja en la ejecución de los programas, donde la diferencia de tiempo se vuelve evidente al trabajar con arreglos más grandes.
- También se evidenció que la elección del pivote y la forma en que se divide el problema afectan directamente el rendimiento de QuickSort. Cuando el particionado es equilibrado, el algoritmo es muy eficiente; cuando no, su comportamiento puede deteriorarse y acercarse a $O(n^2)$.
- Como conclusión general, el análisis de estos algoritmos permite entender que la eficiencia no depende solo de la complejidad matemática, sino también de la estructura de los datos y del contexto de uso. Por ello, la elección entre MergeSort y QuickSort debe hacerse según la necesidad de estabilidad, memoria disponible y tipo de entrada.